

4. Write a C program where a parent process creates multiple child processes and synchronizes their completion using `wait()` and `waitpid()`. Compare the behavior of both functions.

```
knsat@SRITHIKA: ~  
GNU nano 8.7.1 state  
#include <stdio.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
int main() {  
    pid_t pid = fork();  
  
    if (pid == 0) {  
        printf("Child Stage 1 - PID: %d, PPID: %d\n", getpid(), getppid());  
        sleep(5);  
        printf("Child Stage 2 - Waking up\n");  
    } else if (pid > 0) {  
        printf("Parent Stage 1 - PID: %d\n", getpid());  
        wait(NULL);  
        printf("Parent Stage 2 - Child done\n");  
    }  
    return 0;  
}
```

```
knsat@SRITHIKA:~$ nano states.c  
knsat@SRITHIKA:~$ nano sync.c  
knsat@SRITHIKA:~$ gcc states.c -o states  
knsat@SRITHIKA:~$ ./states  
Parent Stage 1 - PID: 3592  
Child Stage 1 - PID: 3593, PPID: 3592  
Child Stage 2 - Waking up  
Parent Stage 2 - Child done  
knsat@SRITHIKA:~$ ps -l  
F S  UID      PID      PPID  C  PRI  NI ADDR SZ WCHAN  TTY          TIME CMD  
4 S  1000      745      738  0   80   0 -  1567 do_wai pts/0        00:00:00 bash  
0 R  1000     3603      745  0   80   0 -  1798 -      pts/0        00:00:00 ps
```